

VERIQO Data Handling Policy

What VERIQO holds, where it lives, how long it stays, and what it cannot do. Written for a customer's data protection, security, or procurement review.

Status: this describes the software's actual behaviour. It is not a contractual data processing agreement. A pilot involving personal data requires a DPA negotiated separately; this document is input to that conversation, not a substitute for it.

1. The central fact

VERIQO does not store evidence documents. It stores metadata about them — including a hash the customer computed — while the documents themselves remain in the customer's own storage at a URI the customer controls.

Everything else in this policy follows from that. The data VERIQO holds is a *record about* your evidence, not your evidence.

2. What VERIQO holds

Category	Examples	Notes
Evidence metadata	evidence ID, case ID, tenant ID, SHA-256, URI, filename, media type, byte size	The URI points into customer storage; VERIQO never dereferences it.
Provenance and custody	collector, source, acquisition record, ordered custody events with actor and logical tick	actor and collector frequently identify people . See §6.
Domain metadata	claim ID, policy ID, party ID, vessel identity, port code, document type, holder identity	Customer-supplied business identifiers. May be personal data.
Decisions and actions	outcome, rationale, finding references, authorization, execution receipt	Rationale is free text and may contain whatever the customer writes.
Audit ledger	hash-chained records of decisions, authorizations, executions	Append-only and tamper-evident. See §5 on deletion.
Operational data	request logs (method, path, status, latency), operational counters	No request bodies are logged.

What VERIQO never holds: the evidence bytes; credentials belonging to customer systems; anything VERIQO fetches on its own initiative (it makes no outbound calls to customer storage).

3. Where it lives

- **In memory and on local disk** of the deployment, in write-ahead log segment files.
- **Not encrypted at rest by the application.** Disk-level encryption is a deployment responsibility (encrypted volumes or filesystems) and must be configured by whoever operates the deployment. Stated as a real gap, not implied.
- **In transit:** TLS and mutual TLS are supported but are configured per deployment and are not on by default. Verify for your deployment.

- **No third-party sub-processors** are involved in the software itself. Where a deployment runs (a cloud region, a customer's own datacentre) is a deployment decision that determines actual data residency.
-

4. Retention and legal hold

Every evidence record enters retention governance at finalization — required, not optional, so nothing is finalized while ungoverned.

States: `ACTIVE` → `RETENTION_ELIGIBLE` → `PURGE_ELIGIBLE` → `PURGED`, with `HELD` and `REDACTION_REQUIRED` → `REDACTED` branches.

Legal hold prevents purge on two independent levels: the state machine has no transition from `HELD` to `PURGE_ELIGIBLE`, and the purge operation separately refuses while any hold is present. Placing and releasing a hold are themselves recorded as custody events, so a hold is part of the custodial record rather than metadata beside it.

Retention periods are set per tenant and evidence class. There is no global default that silently deletes anything.

Current limitation: retention policy, holds, and purge are driven through the Go API; no HTTP route exposes them yet. A pilot that needs customer-driven holds should raise this during scoping.

5. Deletion — and its honest limits

Purging removes evidence content from the governed record and marks it `PURGED`. **The audit ledger retains the fact that the item existed and that it was purged.** The ledger is hash-chained: removing a record would break the chain and be detected as tampering. This is deliberate — an evidence system whose audit trail can be silently rewritten is not an evidence system.

The practical consequence, stated plainly for a data protection review: **VERIQO cannot make it as though a record never existed.** It can remove content and record the removal. Where a legal erasure obligation requires the underlying document to be destroyed, that is achievable — the document is in customer storage — and VERIQO will retain the metadata record and the hash unless that metadata is itself purged.

Requests that require erasing the fact of a decision having been made conflict with the system's purpose and should be raised before a pilot begins, not during one.

6. Personal data

VERIQO has no dedicated personal-data classification, no automated subject-access export, and no built-in pseudonymization.

Personal data nonetheless commonly arrives in:

- `collector`, `actor`, `executing_actor`, `ledger_actor` — usually named individuals.
- `party_id`, `holder_identity` — may identify individuals.
- `rationale`, `reason`, `acquisition_record` — free text.
- `filename` — frequently contains names.

Recommendations for a pilot handling personal data:

1. Use role or system identifiers rather than personal names in actor fields where your process permits.

2. Treat free-text fields as customer-controlled: do not write anything into `rationale` that you would be unable to retain.
 3. Complete a DPA and, where required, a DPIA before submitting personal data. The retention/hold/purge mechanics above are the technical controls available to support it.
-

7. Access control

- Data access is **tenant-scoped structurally**: records are keyed by tenant, so a cross-tenant read fails as "not found" before any ownership check runs.
 - Under JWT authentication, the tenant a caller may act as is derived from the verified subject; naming an unauthorized tenant is refused with 403.
 - RBAC exists but the role table is **global, not per-tenant** — a named open item.
 - Operators of the deployment have host-level access to the WAL files. There is no application-level protection against a deployment administrator reading them; that boundary is handled by whoever controls the host.
-

8. Export and portability

- **Dossier packages** (`.zip`) are self-contained and remain verifiable indefinitely with the standalone verifier, with no VERIQO service reachable and no licence in force.
 - **Full tenant record**: a WAL backup can be taken and handed over.
 - **On pilot termination**, request a backup before shutdown. The evidence bytes were always yours; the exported packages remain usable regardless.
-

9. Logging

Request logs record method, path, status, and latency only — **no request bodies**. The request path is explicitly escaped before logging to prevent log injection. Logs go to the deployment's stdout and are handled by whatever collects them, which is a deployment concern.

10. Breach posture

If the deployment were compromised, an attacker would obtain metadata, hashes, decisions, and the audit ledger — **not** the evidence documents. That metadata may itself be sensitive (claim identifiers, party names, vessel identities, filenames), so this reduces but does not eliminate impact.

Tamper-evidence is a genuine control here: an attacker who alters the ledger breaks its hash chain, and verification detects it. An attacker can destroy or truncate; they cannot substitute a plausible alternative history that passes verification.

See `docs/customer/VERIQO_INCIDENT_RESPONSE_PROCEDURE.md`.